

# Privacy Router

A Sovereignty-First Privacy Layer for AI Agents

GenAI & Blockchain · Term Project · June 2026

## HOW IT WORKS

**Extractor** → **Judge** → **Router**

Detects sensitive information in prompts, decides whether to mask or process locally, then routes to the appropriate model.

Local (Ministral 3B)

Cloud (Gemini Flash Lite)

Mask & Send

# Problem

AI agents send sensitive data to external LLMs

## PROBLEM

### AI agents send sensitive data to external LLMs

Existing solutions (OpenAI Privacy Filter) focus on **English/US PII** (SSN, ZIP). They cannot understand Korean RRN, +82 phone numbers, or research context. The only options are **share all** or **block all**.

## EXAMPLE

### RRN / PII Leakage

PII such as Korean RRN (13-digit national ID) slips past existing LLM guardrails. Regex **cannot understand context**.

## EXAMPLE

### Research Secret Leakage

Unpublished research ideas, lab budgets, submission plans — **competitive intelligence** is undetectable by regex.

## CONCLUSION

### Privacy Router: context-aware prompt firewall

Detect, mask, route — sensitive data stays local, only safe queries reach the cloud. **Data sovereignty for all users.**



# Target Users

AI prompts frequently contain sensitive data, yet existing security solutions focus mainly on English PII.



## Korean University Researchers

They input unpublished paper ideas, experimental designs, and IRB-related data into LLMs. **Protecting research competitiveness** is critical.



## Professionals & Corporate Researchers

They generate prompts containing trade secrets, internal decisions, and budget data. **Corporate secrecy protection** is essential.



## Individual Users

Resident Registration Numbers, phone numbers, emails — **PII** gets casually included in everyday prompts.

### KEY INSIGHT

Existing solutions (OpenAI Privacy Filter) focus on **English / US PII (SSN, ZIP)**. No system handles Korean RRN, +82 phone numbers, or research secrets.

# Architecture

How a user prompt flows through the Privacy Router pipeline





# Extractor

Socratic method-based detection — Socratic sensitivity detection with SCREAMING\_CASE categories, language-agnostic rules with examples like Korean RRN

PII

## Personally Identifiable Info

RRN, phone numbers, emails, passport numbers, name + affiliation pairs

"901212-1234567" → RESIDENT\_REGISTRATION\_NUMBER

"010-1234-5678" → CONTACT\_NUMBER

Research

## Research Secrets

Unpublished research, internal decisions, strategies, manufacturing plans

"new attention mechanism idea"

→ UNPUBLISHED\_RESEARCH\_DATA

Business

## Business Secrets

Credentials, internal URLs, budgets, salary data

"API key sk-proj-..." → API\_CREDENTIAL

"budget 500M KRW" → BUDGET\_INFORMATION

Tags use **free-form SCREAMING\_CASE** — generated contextually by the SLM using Socratic reasoning, not from a hardcoded list. Korean-specific detection rules for RRN, phone numbers, research secrets, and business secrets.

# Judge

Single-Axis Masking Test — "If I replace every sensitive span with [REDACTED], does the user's request still make sense?"

## SINGLE-AXIS MASKING TEST

**"If I replace every sensitive span with [REDACTED], does the user's request still make sense?"**

Verb heuristic determines action: **Creation** → mask\_and\_send, **Consultation** → mask\_and\_send, **Interrogation** → process\_locally



Yes → **Mask & send to cloud**



No → **Process locally**

*// Core concept: Single-Axis Masking Test*

`SENSITIVE_SPANS` replaced with `[REDACTED]` → request still makes sense? → `mask_and_send`

`SENSITIVE_SPANS` replaced with `[REDACTED]` → request broken? → `process_locally`



# Router

Pure Execution Layer (No LLM) — Decision table, UID-based masking, hydration

## ROUTE 1

### External API

Replace sensitive spans with placeholders → send to cloud LLM → restore originals in response

`mask_text()` → Cloud LLM → `hydrate_text()`

## ROUTE 2

### Local API

When the prompt itself is about sensitive data. **Entire processing stays local.**

`process_locally()` → Local LLM

## ROUTE 3

### Ask to user

When the system cannot decide. **Asks the user for confirmation.**

`409` → `needs_input` → user chooses

## MASKING CONTRACT

Deterministic masking: **UID-based placeholders** (e.g. `RESIDENT_REGISTRATION_NUMBER#1`) — no brackets. Originals stored encrypted in SQLite. The mask → LLM → restore cycle ensures originals never reach the cloud.

# Detection Examples

Real detection results — contextual understanding is the core strength

## RESEARCH IDEA DETECTION

**Input:** "I'm designing a novel loss function that outperforms contrastive learning for multi-modal alignment."

NOVEL\_LOSS\_FUNCTION

UNPUBLISHED\_RESEARCH\_DATA

→ **mask\_and\_send** (contextual — no trigger keywords)

## BUSINESS STRATEGY DETECTION

**Input:** "We're planning to acquire StartupX for \$50M and integrate their proprietary tokenizer into our next release."

ACQUISITION\_PLAN

FINANCIAL\_FIGURE

PRODUCT\_INTEGRATION

→ **mask\_and\_send**

## PII DETECTION

**Input:** "My RRN is 901212-1234567."

RESIDENT\_REGISTRATION\_NUMBER

→ **mask\_and\_send** (pattern-based)

**Key:** Contextual detection is the real strength — the SLM understands *meaning*, not just patterns. Tags are free-form SCREAMING\_CASE, generated by Socratic reasoning. Confidence: 0.88–0.98



# Two-Phase Extraction

Phase 1: Socratic extraction — Phase 2: Critic pass finds what Phase 1 missed

## TWO-PHASE EXTRACTOR

**Phase 1:** Socratic extraction generates SCREAMING\_CASE sensitivity categories.

**Phase 2:** Critic pass reviews results and fills gaps. Deduplication and hallucination filtering merge the final output.

## V2 RESULTS

**1 → 4**   **\$0**   **+4**  
Models hitting 7/7   Added cost   Improved cases

## V1 — PATTERN MATCHING

### Keyword-Dependent

Relies on explicit cues: "registration number", "decision", "adoption".

**Result:** Only Gemini Flash Lite detects business/research secrets. All open-source models: 0%.

## V2 — CONTEXTUAL REASONING

### Question-Based Inference

The Extractor applies two contextual reasoning questions to every sentence:

*"If this sentence appeared in tomorrow's newspaper, would a competitor gain an advantage?"*

*"If disclosed before publication, would the researcher be harmed?"*

These questions enable the SLM to detect sensitive information **without relying on keywords**. The model understands *meaning* — a sentence like "TSMC 3nm process adoption" is classified as a business secret even though no explicit keyword like "secret" or "confidential" appears.



# Cost Analysis

\$0.19/month — cost-efficient Two-Tier routing

\$0.19

Est. monthly cost

46

Prompts over 7 days

31

Sensitive detected

9

Processed locally

## TWO-TIER MODEL STACK

### Cost-Performance Balance

| MODEL                 | \$/1M TOK | V2 SCORE |
|-----------------------|-----------|----------|
| Minstral 3B           | \$0.10    | 7/7      |
| Gemini 3.1 Flash Lite | \$0.25    | 7/7      |
| DeepSeek v4 Flash     | \$0.10    | 7/7      |
| Gemma 4 26B-A4B       | \$0.06    | 7/7      |

## BEST COST-PERFORMANCE

### Gemma 4 26B-A4B

\$0.06/1M tok — achieves the same 7/7 full detection as Gemini Flash Lite (\$0.25) at 4x lower cost.

# Cost-efficiency ranking

- 1. Gemma 4 26B-A4B → \$0.009/case
- 2. Minstral 3B → \$0.014/case
- 3. DeepSeek v4 → \$0.014/case
- 4. Gemini Flash Lite → \$0.036/case



# Privacy & Security

Data flow and threat model

## DATA FLOW

### Sensitive Data Stays Local

- Raw text → **processed only by local SLM**
- Only masked text sent to cloud LLM
- Originals **encrypted in SQLite**
- Restoration performed locally only

## THREAT MODEL

### Guaranteed Security Properties

- ✓ **Zero sensitive data leakage** — no external transmission without masking
- ✓ **Deterministic masking** — same input → same masking (reproducible)
- ✓ **Auditable** — every decision logged
- ⚠ **SLM hallucination risk** — detection model may miss → mitigated by Critic pass

## CORE SECURITY PRINCIPLE

**"Only masked data reaches the cloud"** — Privacy Router structurally prevents sensitive data from reaching external LLMs. No sensitive information leaves the system without user consent.

# 7-Day Usage Log

Real usage data — 46 prompts, 31 sensitive detected, 22 masked, 9 processed locally

46

Total prompts

15

Safe (sent externally)

22

Masked & sent

9

Processed locally

## Routing Distribution



|                      |                      |              |                |
|----------------------|----------------------|--------------|----------------|
| DAY 1                | DAY 2                | DAY 3        | DAY 4-7        |
| General conversation | Email drafting (PII) | Paper review | Research tasks |
| 5 prompts            | 7 prompts            | 6 prompts    | 28 prompts     |

**Key finding:** Over 7 days, **zero leakage** — not a single sensitive prompt was sent to an external API without masking. 67% of all prompts contained sensitive data.



# Tech Stack & Integration

MCP tool + OpenAI-compatible proxy — agent-native privacy

## MCP SERVER

### **process(text, action, model)**

Single unified tool. The **action** parameter controls behavior:

```
auto → Full pipeline (detect → route → generate)
classify → Detection only (no LLM call)
generate → Mask + pass to LLM
allow → Skip detection, direct to LLM
```

## OPENAI COMPATIBLE

### **/v1/chat/completions**

Drop-in OpenAI SDK proxy. **Any agent** can use Privacy Router by changing the base URL. Built with FastAPI + SQLAlchemy, SvelteKit dashboard, litellm for model routing.

```
# Tech: FastAPI + SQLAlchemy (not SQLAlchemy)
# Frontend: SvelteKit
# Routing: litellm
```

## Hermes

Butler agent · MCP direct

## OpenCode

CLI agent · Custom provider

# A lower bound on model size exists for contextual privacy detection.

Gemma 4 26B crossed that threshold —  
at \$0.06/1M tokens.

## SUMMARY

Detection  
Framework

Socratic Sensitivity  
Detection

Prompt Engineering

v1 → v2: 0% → 100%

Optimal Model

Gemma 4 26B · \$0.06/1M

Integration

MCP + OpenAI Proxy

Test Coverage

64 unit + 17 eval

Roadmap

Data → Prompt → Model